

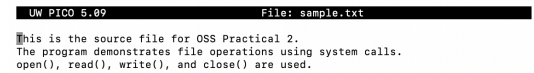
Practical 2

Aim

To develop a C program using the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another and to understand the transition of control between user space and kernel space during file operati

Step 1: Creating the Source File

```
[sahasra@Mac oss_practical2 % nano sample.txt]
[sahasra@Mac oss_practical2 % cat sample.txt]
This is the source file for OSS Practical 2.
The program demonstrates file operations using system calls.
open(), read(), write(), and close() are used.
```



Step 2: Creating the File Copy Program

```
[sahasra@Mac oss_practical2 % nano filecopy.c]
```



Step 3: Compiling and run the program then verify

```
[sahasra@Mac oss_practical2 % nano filecopy.c]
[sahasra@Mac oss_practical2 % gcc filecopy.c -o filecopy]
[sahasra@Mac oss_practical2 % ls]
filecopy      filecopy.c    sample.txt
```

```
[sahasra@Mac oss_practical2 % ls]
filecopy      filecopy.c    sample.txt
[sahasra@Mac oss_practical2 % ./filecopy]
File copied successfully!
[sahasra@Mac oss practical2 % cat copy.txt]
```

```
[sahasra@Mac oss_practical2 % nano filecopy.c]
[sahasra@Mac oss_practical2 % gcc filecopy.c -o filecopy]
[sahasra@Mac oss_practical2 % ls]
filecopy      filecopy.c    sample.txt
[sahasra@Mac oss_practical2 % ./filecopy]
File copied successfully!
[sahasra@Mac oss_practical2 % cat copy.txt]
This is the source file for OSS Practical 2.
The program demonstrates file operations using system calls.
open(), read(), write(), and close() are used.
```

Step 4: Tracing File System Operations Using fs_usage

Since strace is not available in the macOS environment, fs_usage was used to observe file system activity while executing the command cat sample.txt. The trace showed file-related operations such as read() and write(). These operations involve kernel services for reading data and writing the output to the terminal.

```
[sahasra@Sah oss_practical2 % sudo fs_usage -w -f filesystems
0.000001 sudo.1f
3633
14:59:51.462377 read F=7 B=0xf6
0.000001 sudo.1f
3633
14:59:51.462383 write F=6 B=0x400
0.000003 sudo.1f
3633
14:59:51.462385 read F=7 B=0x1ec
0.000001 sudo.1f
3633
14:59:51.462391 write F=6 B=0x400
0.000003 sudo.1f
3633
14:59:51.462394 read F=7 B=0x1ec
0.000001 sudo.1f
3633
14:59:51.462400 read F=7 B=0xf6
```

Step 5: Process Information Using ps

```
[sahasra@Sah oss_practical2 % ps -o pid,ppid,state,command
PID PPID STAT COMMAND
2735 2734 S -zsh
2974 2973 S+ -zsh
```

Step 6: Monitoring Processes Using top

```
sahasra@Sah oss_practical2 % top
```

```
Processes: 692 total, 3 running, 689 sleeping, 3061 threads 15:09:37
Load Avg: 2.15, 2.90, 2.73 CPU usage: 5.97% user, 4.1% sys, 90.0% idle
SharedLibs: 1020M resident, 198M data, 163M linkedit.
MemRegions: 0 total, 0B resident, 229M private, 1689M shared.
PhysMem: 15G used (1684M wired, 2023M compressor), 259M unused.
VM: 294T vsize, 6144M framework vsize, 0(0) swapins, 0(0) swapouts.
Networks: packets: 8379195/2235M in, 279833/198M out.
Disks: 1188477/21G read, 873586/11G written.
```

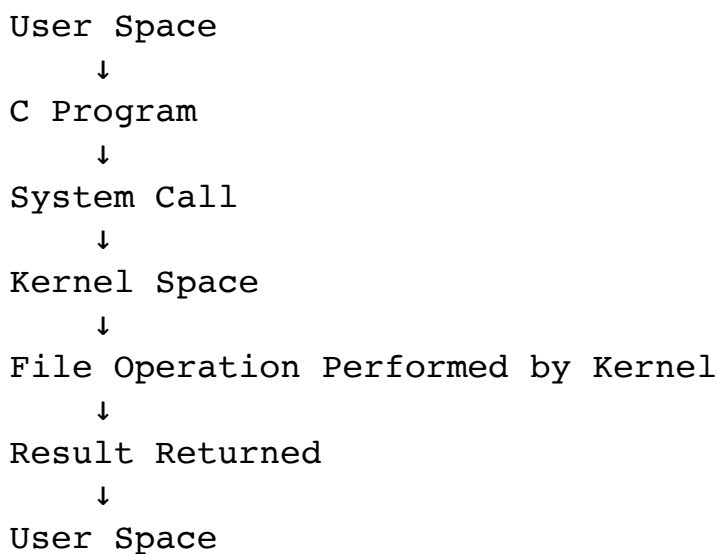
PID	COMMAND	%CPU	TIME	#TH	#WQ	#PORT	MEM	PURG	CMPRS	PGRP
394	WindowServer	41.5	47:16.58	25	6	3158+	488M+	18M-	100M	394
0	kernel_task	11.2	17:57.34	617/10	0	0	16M+	0B	0B	0
3028	top	6.1	00:00.71	1/1	0	29	8112K+	0B	0B	3028
2797	Telegram	5.7	01:00.51	30	5	677-	176M	96K	141M	2797
2647	Google Chrom	5.7	03:47.36	24	1	251	406M	0B	82M	2617
3029	screencaptur	5.2	00:00.24	6/1	3	200	9024K+	704K-	0B	594
403	coreaudiod	3.5	02:03.67	20	12	713-	22M+	0B	9744K	403
326	mediaremoted	2.1	00:02.82	11	8	210	7217K+	0B	3232K	326
2623	Google Chrom	1.7	01:28.68	16	2	307	288M	53M	36M	2617
521	com.apple.Dr	1.6	03:15.06	8	6	1401	31M	0B	4432K	521
495	mDNSResponde	1.6	03:19.72	5	3	131	9857K	0B	1632K	495
610	trustd	1.1	03:09.00	3	2	214	8448K	1168K	3776K	610
397	loginwindow	1.0	00:14.01	6	3	703-	45M+	0B	34M	397
369	locationd	0.9	00:22.83	6	3	307+	10M+	224K	2976K	369

User Space and Kernel Space Transition

The C program initially executes in **user space**. When the program performs file operations using `open()`, `read()`, `write()`, and `close()`, a system call is made to request a service from the operating system.

The system call causes control to transition from **user space to kernel space**. The operating system kernel performs the requested file operation and returns the result to the program. Control then returns from **kernel space back to user space**.

The transition can be represented as:



Result

Thus, a C program was successfully developed and executed to copy the contents of one file to another using the system calls `open()`, `read()`, `write()`, and `close()`. The transition of control between user space and kernel space during system call execution was understood. The **strace** command and the macOS tracing utility **dtruss** were attempted to trace the system calls generated by `cat sample.txt`; however, tracing was restricted in the macOS environment.

